



UNIVERSITÄT
LEIPZIG

Natural Language Processing

Week 2: Corpora and Regular Expressions

Leonie Weissweiler

16.04.2026

Moodle

- Question: were the late registrations automatically enrolled in moodle?
- Self-enrolment password: considerthelilies

Goals of Natural Language Processing

- 1. Aid humans in writing.

Correcting mistakes, formulating and paraphrasing text, transcription.

- 2. Identify texts related to spoken or written requests.

Text information retrieval, semantic text similarity, question answering.

- 3. Make sense of texts without reading the originals.

Categorization, information extraction, summarization, translation.

- 4. Instruct and be advised by a computer.

Audio interfaces (e.g., dialog systems, robotics), learning and assessment.

- 5. Converse with computers as if they were human.

Turing test, Conversational AI and chatbots, argumentation, affective computing.

- 6. Understanding the nature of language and its relation to (artificial) intelligence.

Pragmatics, causality, alignment and grounding, multimodality, simulation, linguistics.

Business Use Cases

- Customer service and customer relation management
E.g., classification of customer support messages, dialogue systems and chatbots on web pages, or template-driven response generation.
- Business analytics and decision support
E.g., sentiment analysis of company- or product-related news or social media, summarization of user feedback, market forecast analysis for investments.
- Legal compliance and moderation of user-generated content
E.g., contract checking, invoice processing, classifying posts wrt. inappropriate content such as sexual messages or hate speech.
- Data journalism (e.g., on leaked documents)
E.g., search, information extraction, summarization.

Governance Use Cases

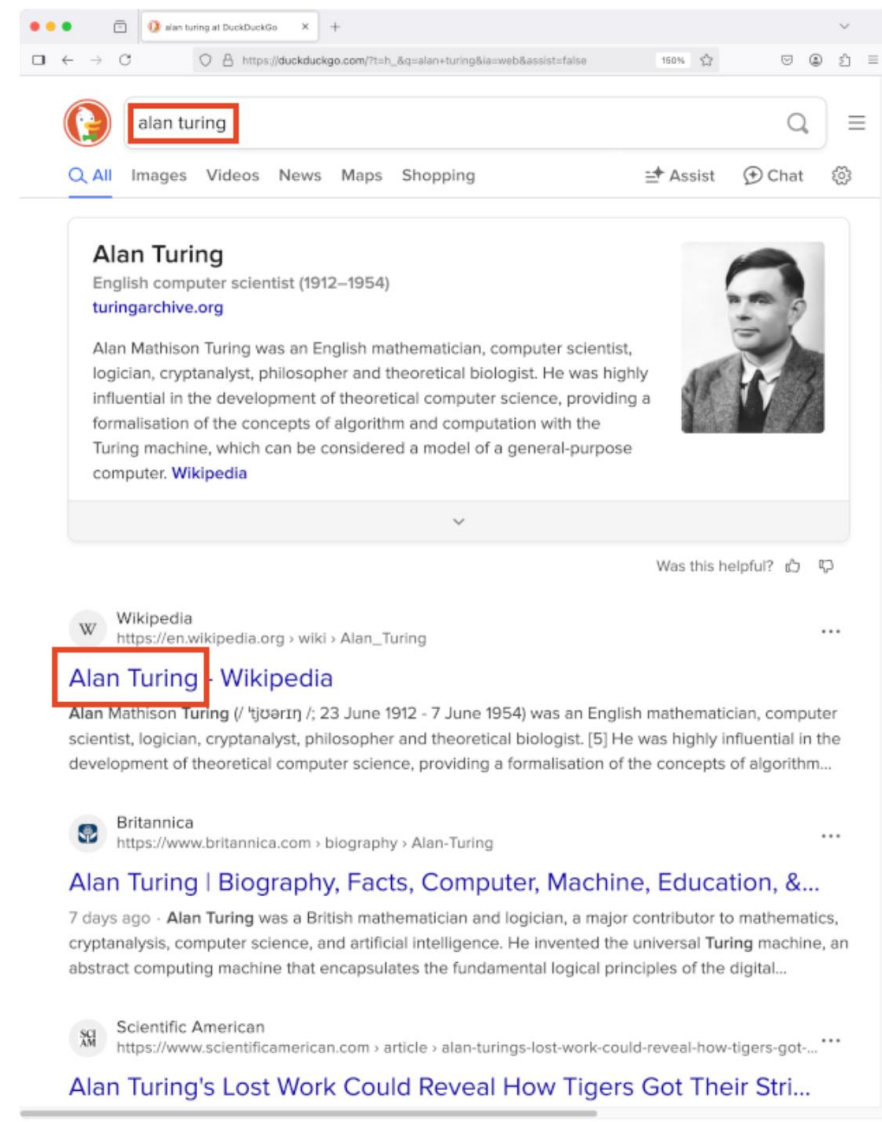
- Public health monitoring
E.g., social media monitoring for flu prediction
- Incident monitoring and disaster management
E.g., social media monitoring for damage assessment
- Criminal investigation and counter terrorism
E.g., style-based authorship analysis; search, classification, and summarisation of the dark web.
- Intelligence and espionage
E.g., monitoring and transcription of communication worldwide.

Types of Tasks

- Analysis: the decoding of structured information from text
- Synthesis: the encoding of (structured) information into text

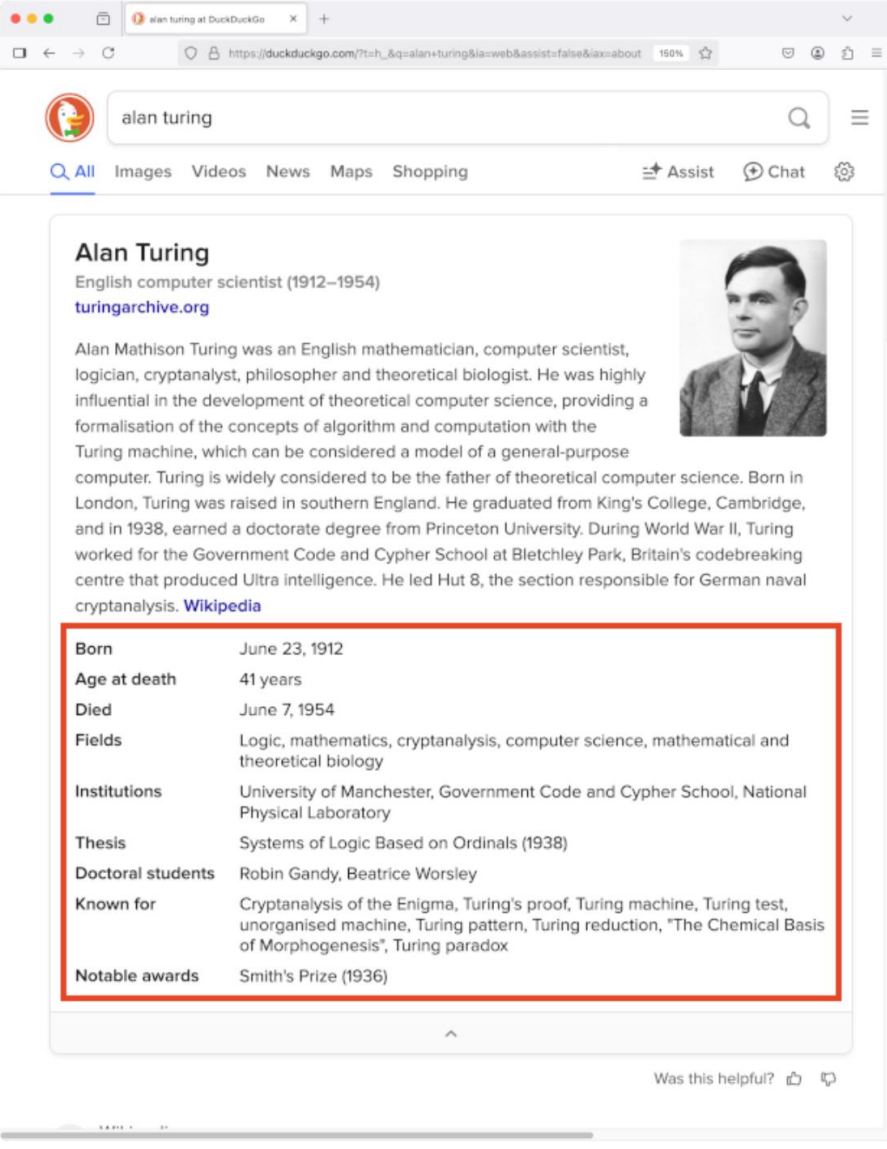
NLP Inside a Search Engine

- Search engines face the vocabulary mismatch problem between users and document authors.
- Users query one way, authors write another.
- Users still expect to find all relevant documents, regardless of how they write.
- Search engines implement a pipeline of basic NLP tasks to address this problem:
 - 1. Tokenization
 - 2. Lemmatization
 - 3. Stemming



NLP Inside a Search Engine

- Search engines return “knowledge panels” along search results.
- This relieves users from browsing the search results for basic facts.
- Unless it is collected manually, knowledge can be extracted from text using this pipeline:
 - 1. Tokenization
 - 2. POS tagging
 - 3. Named entity recognition
 - 4. Reference resolution
 - 5. Entity relation extraction



The screenshot shows a DuckDuckGo search result for "alan turing". The search bar at the top contains "alan turing". Below the search bar, there are tabs for "All", "Images", "Videos", "News", "Maps", and "Shopping". To the right of these tabs are links for "Assist", "Chat", and a settings icon. The main content area features a knowledge panel for Alan Turing. It includes a portrait of Alan Turing, his title "English computer scientist (1912–1954)", and a link to "turingarchive.org". A paragraph of text describes his life and work, mentioning his role in the development of the Turing machine and his work at Bletchley Park. Below this text is a table of key facts about Alan Turing, which is highlighted with a red border. At the bottom right of the panel, there is a link to "Wikipedia".

Born	June 23, 1912
Age at death	41 years
Died	June 7, 1954
Fields	Logic, mathematics, cryptanalysis, computer science, mathematical and theoretical biology
Institutions	University of Manchester, Government Code and Cypher School, National Physical Laboratory
Thesis	Systems of Logic Based on Ordinals (1938)
Doctoral students	Robin Gandy, Beatrice Worsley
Known for	Cryptanalysis of the Enigma, Turing's proof, Turing machine, Turing test, unorganised machine, Turing pattern, Turing reduction, "The Chemical Basis of Morphogenesis", Turing paradox
Notable awards	Smith's Prize (1936)

Part-of-Speech Tagging (now with UD!)

Given a tokenised text, identify the part of speech of each token using Universal POS tags

PROPN	PROPN	PROPN	SYM	NUM	PROPN	NUM	SYM	NUM	PROPN	NUM	SYM	VERB
Alan	Mathison	Turing	(	23	June	1912	-	7	June	1954	)	was
DET	ADJ	NOUN	PUNCT	PRON	VERB	ADV	ADJ					
an	English	mathematician	.	He	was	highly	influential					
ADP	DET	NOUN	ADP	ADJ	NOUN	NOUN	PUNCT					
in	the	development	of	theoretical	computer	science	.					

ADJ: adjective

ADP: adposition

ADV: adverb

AUX: auxiliary

CCONJ: coordinating conjunction

DET: determiner

INTJ: interjection

NOUN: noun

NUM: numeral

PART: particle

PRON: pronoun

PROPN: proper noun

PUNCT: punctuation

SCONJ: subordinating conjunction

SYM: symbol

VERB: verb

X: other

Types of Tasks

- Analysis: the decoding of structured information from text
- Synthesis: the encoding of (structured) information into text

Token n-gram counting

1-gram	Count
Alan	24,000,000
Mathison	110,000
Turing	1,300,000
(	3,300,000,000
23	320,000,000
June	210,000,000
1912	12,000,000
-	19,000,000,000
7	760,000,000
1954	20,000,000
)	3,300,000,000
was	3,700,000,000
an	2,500,000,000
English	270,000,000
mathematician	2,200,000
.	39,000,000,000
He	2,400,000,000
highly	73,000,000
influential	9,300,000
in	15,000,000,000
the	43,000,000,000
development	420,000,000
of	25,000,000,000
theoretical	25,000,000
computer	240,000,000
science	260,000,000

2-gram	Count
Alan Mathison	4,500
Mathison Turing	3,000
Turing (	1,400
(23	330,000
23 June	670,000
June 1912	51,000
1912 -	190,000
- 7	870,000
7 June	550,000
June 1954	100,000
1954)	640,000
) was	41,000,000
was an	41,000,000
an English	6,000,000
English mathematician	36,000
mathematician .	130,000
. He	320,000,000
He was	270,000,000
was highly	1,600,000
highly influential	170,000
influential in	750,000
in the	3,500,000,000
the development	57,000,000
development of	78,000,000
of theoretical	1,500,000
theoretical computer	300,000
computer science	12,000,000
science .	11,000,000

3-gram	Count
Alan Mathison Turing	2,800
Mathison Turing (	52
Turing (23	<42
(23 June	883
23 June 1912	612
June 1912 -	1,400
1912 - 7	92
- 7 June	737
7 June 1954	1,100
June 1954)	1,900
1954) was	109
) was an	9,100
was an English	170,000
an English mathematician	7,600
English mathematician .	1,100
mathematician . He	7,400
. He was	47,000,000
He was highly	110,000
was highly influential	12,000
highly influential in	28,000
influential in the	220,000
in the development	8,500,000
the development of	44,000,000
development of theoretical	19,000
of theoretical computer	30,000
theoretical computer science	310,000
computer science .	540,000



UNIVERSITÄT
LEIPZIG

Questions?

This Week

- Corpora and Zipf's Law
- Formal Grammars
- Regular Expressions



UNIVERSITÄT
LEIPZIG

Corpora

Corpus

Definition: A collection of samples of language use that is authentic, representative, and large.

- Authenticity: Genuine communications; not produced for linguistic analysis; minimum observer disruption.
- Representativeness:
 - Ideal: Approximating the population's distribution; statistical representativeness is practically impossible.
 - Practical: broad coverage and diversity of language varieties; a reasoned balance among varieties.
- Size: Generally: the larger, the more representative. Evaluate how quickly rare phenomena are lost as size shrinks; whether growth broadens variety; trade-off between scale and knowing the contents' properties and quality.

Reference Corpora

- National. Representing the (main) national language(s) of a country.
 - English: British National Corpus (BNC), American National Corpus (ANC)
 - German: The Mannheim German Reference Corpus (DeReKo)
 - Many more: Polish, Czech, Hungarian, Russian, Italian, Modern Greek, ...
- Monitor: Constantly updated to track rapid language change (e.g., neologisms)
 - Bank of English
 - English Trends Corpus
 - Corpus of Contemporary American English (COCA)
 - Wortschatz Leipzig

Language Variety Corpora

- Brown family: Corpora covering language varieties with comparative studies in mind
- Synchronic: Language varieties from the same time period to study their differences
- Diachronic: Language varieties from different time period to study language change

Non-Written Language Corpora

- Spoken: Recordings and transcriptions of spontaneous or planned speech. Coverage of dialects, dialogues, speech fluency, speech disorders, etc. Overlap with reference corpora.
- Sign language. Recordings of sign language use.
- Gestures. Recordings of hand gestures, body positions, facial expressions, gazes.
- Lip reading. Recordings of speakers' lip movements while speaking.

Language Proficiency Corpora

- Developmental. Spoken or written language use by children of various ages.
- Learner. Spoken or written language use by second-language learners.
- Professional. Written language found in professional contexts. Focus on, e.g., news, academic, legal, medical, or parliament contexts. Overlap with reference corpora.

More Categories of Corpora

- Annotated Corpora: parts-of-speech, lemmas, syntax trees, named entities etc.
- Parallel. Texts aligned across languages at the word or sentence level. [OPUS]
 - NLP uses include the training and evaluation of machine translation technology.
 - CL uses include lexical and idiomatic analyses.

The British National Corpus (BNC)

Spoken Language

Genre	Samples	Words
Conversation	30	1 017 025

Written Language

Genre / Subgenre		Topic area	Samples	Words
Academic	—	Humanities/Arts	7	224 872
		Medicine	2	89 821
		Natural Science	6	215 549
		Politics/Law/Education	6	195 836
		Social Science	7	209 645
		Technology/Engineering	2	77 533
Fiction	Prose	—	25	1 010 279
	Broadsheet	Arts	9	36 603
Newspapers	Other	Commerce	7	64 162
		Editorial	1	8 821
		Miscellaneous	25	121 194
		Report	3	48 190
		Science	5	18 245
		Social	13	34 516
		Arts	3	43 687
		Commerce	5	89 170
		Report	7	232 739
		Science	7	13 616
	Tabloid	—	1	121 252

International Corpus of English (ICE)

Spoken Language

Situation / Genre / Sub-genre		Samples
Dialogues	Private	Face-to-face conversations 90
		Phone calls 10
	Public	Classroom lessons 20
		Broadcast discussions 20
		Broadcast interviews 10
		Parliamentary debates 10
		Legal cross-examinations 10
		Business transactions 10
Monologues	Unscripted	Spontaneous commentaries 20
		Unscripted speeches 30
		Demonstrations 10
		Legal presentations 10
	Scripted	Broadcast news 20
		Broadcast talks 20
		Non-broadcast talks 10

Every sample consists of 2,000 words

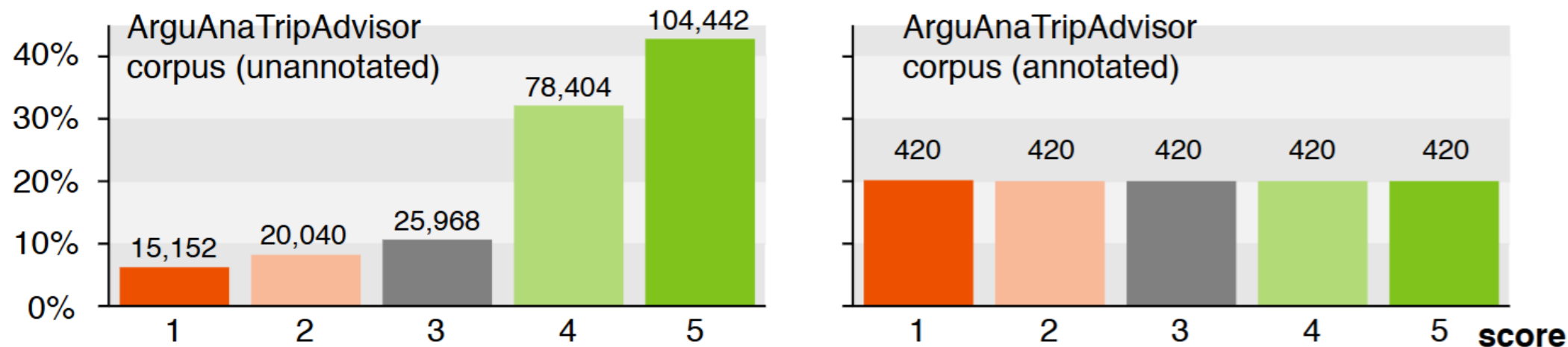
Written Language

Situation / Genre / Sub-genre		Samples
Non-print.	Student writing	Student essays 10
		Exam scripts 10
	Letters	Social letters 15
		Business letters 15
	Academic writing	Humanities 10
		Social Sciences 10
		Natural Sciences 10
		Technology 10
Printed	Popular writing	Humanities 10
		Social Sciences 10
		Natural Sciences 10
		Technology 10
	Reportage	Press news reports 20
		Instructional writing 10
	Persuasive writing	Administrative writing 10
		Skills / Hobbies 10
	Creative writing	Press editorials 10
		Novels and short stories 10

Representativeness and Balance

- “extent to which a sample includes the full range of variability in a population” [Biber 1993] Here: sample is our corpus, population is all of the language variety.
- “A corpus is thought to be representative of the language variety it is supposed to represent if the findings based on its contents can be generalized to the said language variety.” [Leech 1991]
Question: when the corpus has certain features, will we find the same features in further data of that type?
- What is representative to the users of a language? “According to claims, the most likely document that an ordinary English citizen will cast his or her eyes over is The Sun newspaper” [Sinclair 2004] Keyword: reception versus production
- Corpus representativeness is important for generalization, since the corpus governs what can be learned about a given domain.

Representativeness and Balance



- A corpus is representative for a classification C , if it includes the full range of variability of texts with respect to C .
- The distribution of texts over C should be representative for the real distribution.
- “Balance” is ambiguous: resemblance of the true distribution of C (corpus linguistics) vs. equal amounts for each class in C (machine learning).
- Evenly distributed text across C are often favourable for machine learning.

Corpus Linguistics

Corpus linguistics is the investigation of linguistic phenomena on the basis of linguistic corpora in terms of their conditional distribution in them.

Building corpora to:

- Study a language phenomenon at large (exploration). Representativeness (diversity) and size of the corpus are key.
- Study a language phenomenon in comparison to another, or an external variable. Variables can be author demographics, time, cultural context, etc.

Process:

1. Annotation. Manual discovery and coding of the language phenomena, if necessary.
2. Data Retrieval. Subsampling of data relevant to the phenomena / variable in question.
3. Analysis. Descriptive and inferential statistics, and (comparative) visualization.

Concordance using keyword in context

st sight to take an unconscionably long	time	. A common fallacy is the attempt to as
s with Arsenal . ' Graham reckons it 's	time	his side went gunning for trophies agai
ough I did n't , he he , I did n't have	time	to ask him what the hell he 'd been up
was really impressed . I think the last	time	I came I had erm a chicken thing . A ch
away . No Ann would have him the whole	time	. Yeah well [unclear] Your mum would n'
arch 1921 . He had been unwell for some	time	and had now gone into a state of collap
te the planned population in five years	time	. So what are you gon na multiply that
tempt to make a coding time and content	time	the same) . 10 Conclusion I have stres
hearer and for the analyst most of the	time	. Most of the time , things will indeed
in good faith and was reasonable at the	time	it was prepared . The bank gave conside
he had something on his mind the whole	time	. ' ` Perhaps he was thinking of his wr
ctices of commercial architects because	time	and time again they come up with the go
nyway . ' From then on Augusto , at the	time	an economist with the World Bank , and
This may be my last free night for some	time	. ' ` I do n't think they 'd be in any
two reasons . Firstly , the passage of	time	provides more and more experience and t
go . The horse was racing for the first	time	for Epsom trainer Roger Ingram having p
imes better and you would do it all the	time	, right Mm I mean basically you say the
ther , we 'll see you in a fortnight 's	time	. ' ` Perhaps then , ' said Viola , who
granny does it and she 's got loads of	time	. She sits there and does them twice as

Collocations

L3		L2		L1		Keyword	R1		R2		R3	
for	335	the	851	the	1032	time	.	950	the	427	.	294
.	322	at	572	this	380	time	,	661	?	168	,	255
at	292	a	361	first	320	time	to	351	i	141	the	212
,	227	all	226	of	242	time	of	258	.	137	to	120
a	170	.	196	same	240	time	and	223	and	118	a	118
the	130	by	192	a	239	time	for	190	you	104	it	112
it	121	,	162	long	224	time	in	184	it	102	was	107
to	100	of	154	some	200	time	i	177	a	96	and	92
and	89	for	148	last	180	time	he	136	he	92	i	86
in	89	it	117	every	134	time	?	122	,	91	you	76
was	85	in	93	in	113	time	you	120	was	87	in	75
is	78	?s	68	that	111	time	when	118	had	80	?	71
?s	68	and	65	what	108	time	the	90	but	70	of	64
have	59			next	83	time	we	88	to	69	?s	59
that	58			any	72	time	is	85	?	64	is	59
had	55			one	65	time	as	78	she	58	?	58
?	52			?s	64	time	it	70	they	57	he	58

Frequency Distributions

Find the best synonym

Use the hash sign in front of a word to check which of its synonyms are commonly written.

waiting * #response	i	x
waiting for an answer	110,000	35%
waiting for a reply	71,000	22%
waiting for a response	59,000	18%
waiting for reply	15,000	4.6%
waiting for your reply	13,000	4.1%
waiting for the answer	12,000	4.0%
waiting for response	10,000	3.2%
waiting to answer	9,600	3.0%
waiting for your answer	7,500	2.3%
waiting for his answer	7,300	2.3%
waiting for my answer	6,400	2.0%

Google Books Ngram Viewer

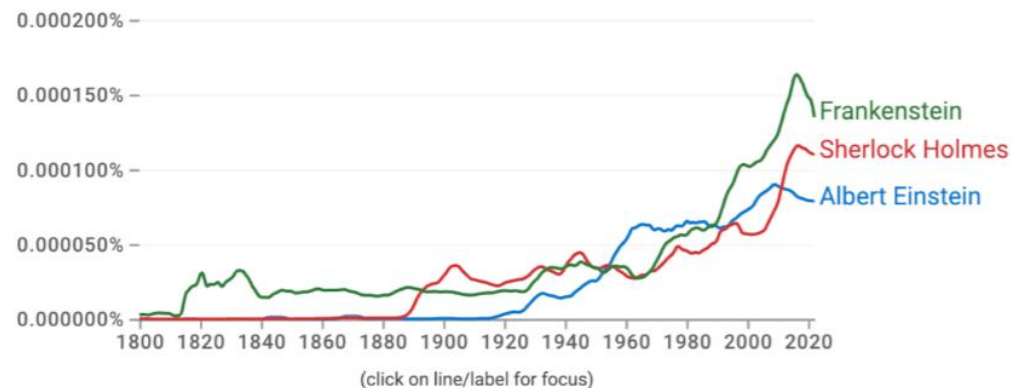
Albert Einstein, Sherlock Holmes, Frankenstein

1800 - 2022

English

Case-Insensitive

Smoothing



Questions

- Can we build the final corpus?
- How much text will cover the vocabulary of a language?
- How large should a corpus be?
- Is random sampling of texts sufficient?

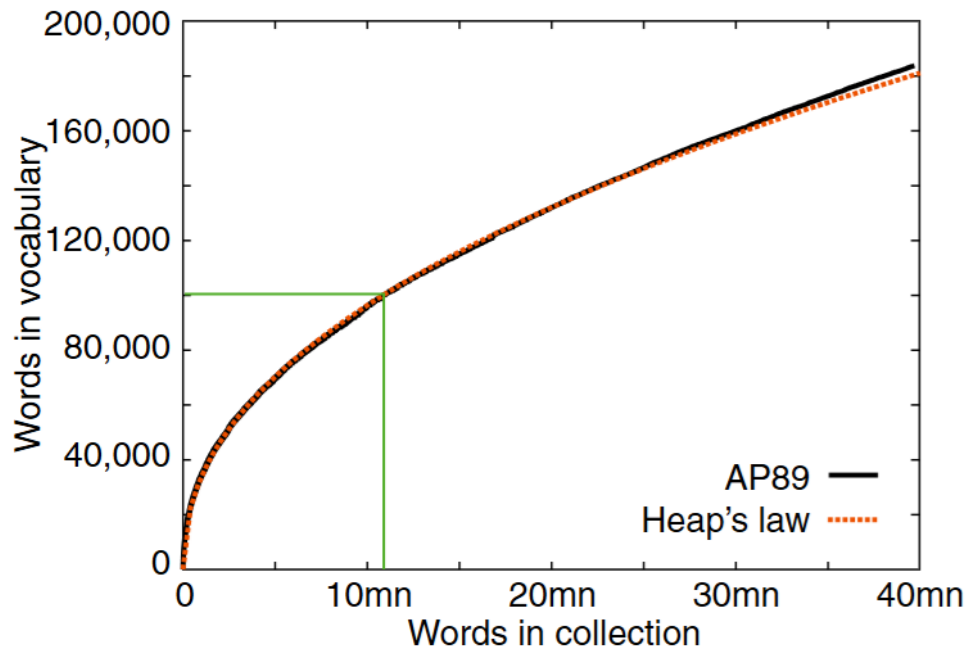
Remarks

- The Zipfian nature of word frequency distributions causes **data sparsity problems**. No matter how large a corpus is, most of the words occurring in it have very low frequency and a small set of frequent words constitutes the large majority of the tokens in the corpus. The distribution of bigrams and linguistic units larger than the word is even more skewed. Even large corpora do not sample the whole vocabulary of the language they represent.
- For example, a trained lexicographer will need to inspect at least 20 instances of an unambiguous word to get an idea of its behaviour. Even in a large corpus such as the (written) BNC, a lexicographer will find that less than 14% of the words have a frequency of 20 or higher.
- Speech synthesis researchers often accept poor modelling of rare words (and other relevant units) in virtue of the fact that they are rare. However, because of The Zipfian nature of linguistic data, although each rare unit has a very low probability to occur, **the overall probability that at least one rare unit will occur in a sentence approaches certainty**.

- An important consequence of the fact that even large corpora are not sampling the full vocabulary of types they are drawn from is that the standard method of estimating the probability of occurrence of a word (or n-gram) by its relative frequency in a corpus is very inaccurate. Statistical language models address this shortcoming with **smoothing**.
- Another consequence of the fact that V keeps growing with corpus size is that **we cannot use it as a measure of lexical richness when comparing corpora of different sizes**. Statistical models of word frequency distributions provide formulas for the expectation (mean) and variance of quantities such as vocabulary size at arbitrary sample sizes and allow the comparison of corpora of different sizes.
- **Word frequencies require specialized statistical models**, since some crucial aspects of standard methods, such as the assumption that the central limit theorem guarantees the normality of sample averages for reasonably large samples, are not appropriate for the extremely skewed word/n-gram frequency data.
- The fact that we can expect words in pretty much any natural language text to have this distribution (and the coefficient α to be close to 1) has found **many applications**, ranging from index compression to term weighting in information retrieval, to cryptography, to Bayesian modelling of morpheme frequencies.

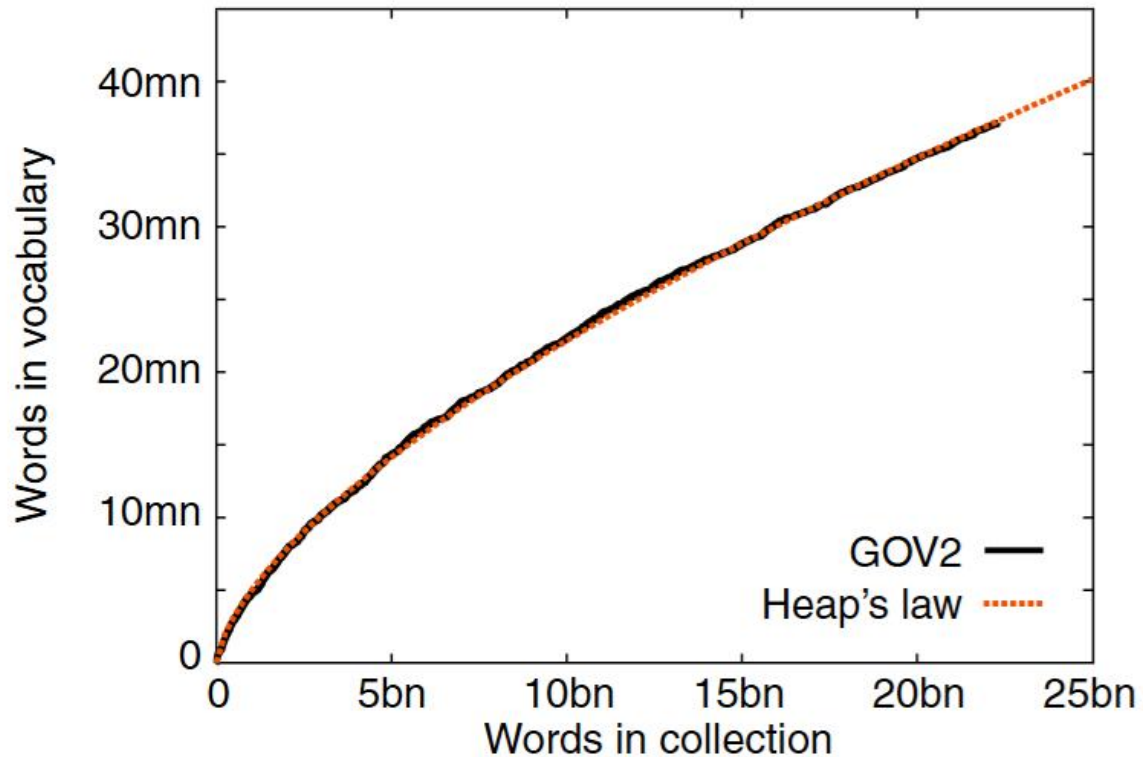
Vocabulary Growth: Heaps' Law

The vocabulary V of a collection of documents grows with the collection. Vocabulary growth can be modelled with Heaps' Law: $|V| = k \cdot n^\beta$, where n is the number of non-unique words (i.e., collection length in words), and k and β are collection parameters (typically, $10 \leq k \leq 100$ and $\beta \approx 0.5$).



- Corpus: AP89
- $k = 62.95$, $\beta = 0.455$
- At 10, 879, 522 words:
 - 100, 151 predicted
 - 100, 024 actual
- At $< 1, 000$ words: poor predictions

Heaps' Law in a Large Corpus



- Corpus: GOV2
- $k = 7.34$, $\beta = 0.648$
- Vocabulary continuously grows in large collections
- New words include spelling errors, invented words, code, other languages, email addresses, etc.

Term Frequency: Zipf's Law

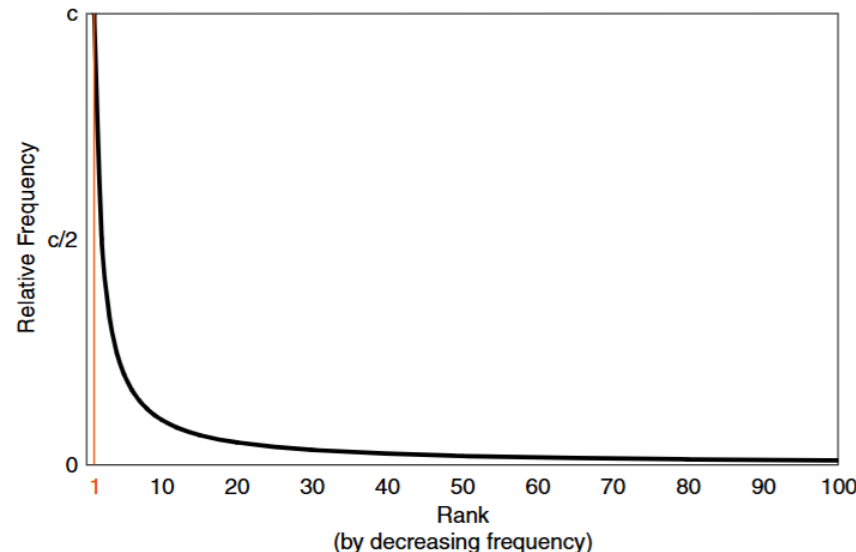
- The distribution of word frequencies is very skewed: Few words occur very frequently, many words hardly ever.
- For example, the two most common English words (the, of) make up about 10% of all word occurrences in text documents. In large text samples, about 50% of the unique words occur only once.
- George Kingsley Zipf, an American linguist, was among the first to study the underlying statistical relationship between the frequency of a word and its rank in terms of its frequency, formulating what is known today as Zipf's law.

Term Frequency: Zipf's Law

The relative frequency $P(w)$ of a word w in a sufficiently large text (collection) inversely correlates with its frequency rank $r(w)$ in a power law:

$$P(w) = \frac{c}{r(w)^a} \quad \Leftrightarrow \quad P(w) \cdot r(w)^a = c,$$

where c is a constant and the exponent a is language-dependent; often $a \approx 1$.



Zipf's Law Example

Top 50 most frequent words from AP89. Have a guess at c ?

r	w	frequency	$P \cdot 100$	$P \cdot r$
1	the	2,420,778	6.09	0.061
2	of	1,045,733	2.63	0.053
3	to	968,882	2.44	0.073
4	a	892,429	2.25	0.090
5	and	865,644	2.18	0.109
6	in	847,825	2.13	0.128
7	said	504,593	1.27	0.089
8	for	363,865	0.92	0.073
9	that	347,072	0.87	0.079
10	was	293,027	0.74	0.074
11	on	291,947	0.73	0.081
12	he	250,919	0.63	0.076
13	is	245,843	0.62	0.080
14	with	223,846	0.56	0.079
15	at	210,064	0.53	0.079
16	by	209,586	0.53	0.084
17	it	195,621	0.49	0.084
18	from	189,451	0.48	0.086
19	as	181,714	0.46	0.087
20	be	157,300	0.40	0.079
21	were	153,913	0.39	0.081
22	an	152,576	0.38	0.084
23	have	149,749	0.38	0.087
24	his	142,285	0.36	0.086
25	but	140,880	0.35	0.089

r	w	frequency	$P \cdot 100$	$P \cdot r$
26	has	136,007	0.34	0.089
27	are	130,322	0.33	0.089
28	not	127,493	0.32	0.090
29	who	116,364	0.29	0.085
30	they	111,024	0.28	0.084
31	its	111,021	0.28	0.087
32	had	103,943	0.26	0.084
33	will	102,949	0.26	0.085
34	would	99,503	0.25	0.085
35	about	92,983	0.23	0.082
36	i	92,005	0.23	0.083
37	been	88,786	0.22	0.083
38	this	87,286	0.22	0.083
39	their	84,638	0.21	0.083
40	new	83,449	0.21	0.084
41	or	81,796	0.21	0.084
42	which	80,385	0.20	0.085
43	we	80,245	0.20	0.087
44	more	76,388	0.19	0.085
45	after	75,165	0.19	0.085
46	us	72,045	0.18	0.083
47	percent	71,956	0.18	0.085
48	up	71,082	0.18	0.086
49	one	70,266	0.18	0.087
50	people	68,988	0.17	0.087

For English: $c \approx 0.1$.

Estimating c for Zipf's Law

For relative frequencies, c can be estimated as follows:

$$1 = \sum_{i=1}^{|V|} P(w_i) = \sum_{i=1}^{|V|} \frac{c}{r(w_i)} = c \sum_{i=1}^{|V|} \frac{1}{r(w_i)} = c \cdot H_{|V|}, \quad \leadsto \quad c = \frac{1}{H_{|V|}} \approx \frac{1}{\ln(|V|)}$$

where $|V|$ is the size of the vocabulary V , and $H_{|V|}$ is the $|V|$ -th harmonic number.

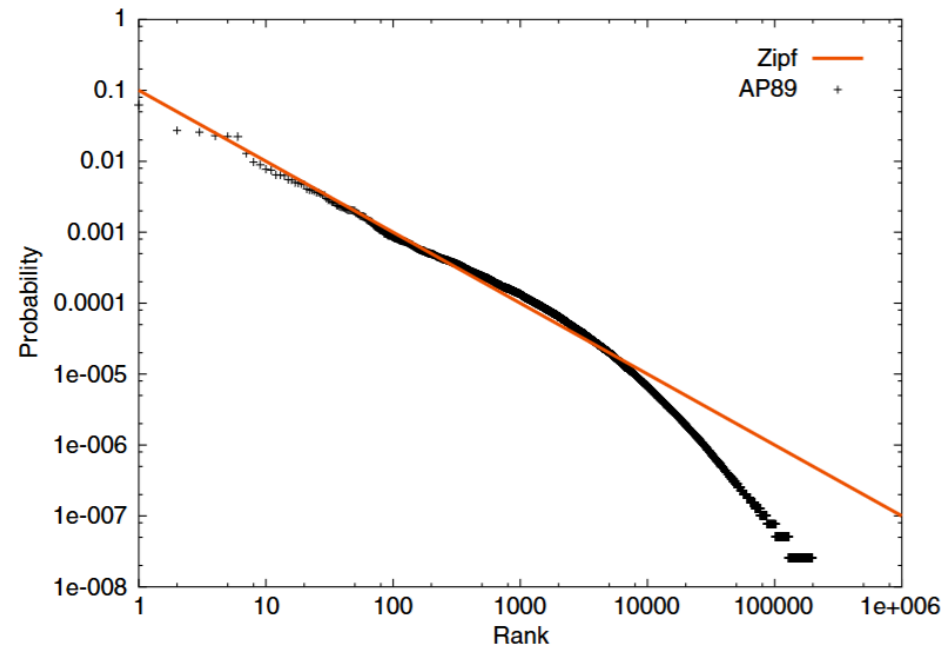
Constant c is language-dependent; e.g., for German $c = 1/\ln(7,841,459) \approx 0.063$. [Wortschatz Leipzig]

Interpreting the relative frequencies as term occurrence “probabilities”, the expected average number of occurrences of a word w in a document d of length m is $m \cdot P(w)$

Zipf's Law Logarithmically

We can obtain a linear form by logarithmisation, yielding a straight line in a plot:

$$\log(P(w)) = \log(c) - a \cdot \log(r(w))$$



Remarks

- As with all empirical laws, **Zipf's law holds only approximately**. While mid-range ranks of the frequency distribution fit quite well, this is less so for the lowest ranks and very high ranks (i.e., very infrequent words). The Zipf-Mandelbrot law is an extension of Zipf's law that provides for a better fit.
- Interestingly, this relation cannot only be observed for words and letters in human language texts or music score sheets, but for **all kinds of natural symbol sequences** (e.g., DNA). It is also true for randomly generated character sequences where one character is assigned the role of a blank space. [Li 1992]
- Independently of Zipf's law, a special case is **Benford's law**, which governs the frequency distribution of first digits in a number.

Words with a Given Frequency

The number of words in V with a given absolute frequency x can be estimated by

$$\frac{1}{x(x+1)}.$$

Derivation: Let r_x denote the lowest rank of a word with frequency x (i.e., the one with the largest rank “number”). Then the number of words with that frequency is derived as follows:

$$r_x - r_{x+1} = \frac{c_1}{p_x} - \frac{c_1}{p_{x+1}} = \frac{c_1 n}{x} - \frac{c_1 n}{x+1} = \frac{c_1 n}{x(x+1)},$$

where $r_x \cdot p_x = c_1$, and $p_x = x/n$ its relative frequency given the total number n of non-unique words. Its proportion of V is obtained by dividing by $|V| = r_1 = c_1 n$.

Observations:

- Estimations are fairly accurate for small x
- Roughly half of all words can be expected to be unique



UNIVERSITÄT
LEIPZIG

Questions?



UNIVERSITÄT
LEIPZIG

Formal Grammars

Terminology

- Alphabet Σ . An alphabet Σ is a finite non-empty set of letters or symbols.
- Word w . A word w is a finite sequence of symbols from Σ . The length of a word $|w|$ is the number of its symbols. ε denotes the empty word; it is the only word with length 0. Σ^* denotes the set of all words over Σ .
- Language L . A language L is a set of words over an alphabet Σ .
- Grammar G . A grammar G is a calculus for defining a language – that is, a set of rules that can be used to derive words. The language belonging to G consists of all derivable, terminal words.

Definition: Grammar

A grammar is a quadruple $G = (N, \Sigma, P, S)$ with

N = finite set of non-terminal symbols

Σ = finite set of terminal symbols, $N \cap \Sigma = \emptyset$

P = finite set of productions or rules

$P \subset (N \cup \Sigma)^* N (N \cup \Sigma)^* \rightarrow (N \cup \Sigma)^*$

S = start symbol, $S \in N$

Remarks

- A rule consists of a left side (premise) and a right side (conclusion), each of which is a word consisting of terminals and non-terminals. The left side must contain at least one non-terminal, and unlike the left side, the right side can also be the empty word. [Wikipedia]
- A rule can be applied to a word consisting of terminals and non-terminals, whereby any occurrence of the left side of the rule in the word is replaced by the right side of the rule: $w \rightarrow w'$.
- Given the rule $w \rightarrow w'$, then w, w' are in the so-called transitive relation $\rightarrow_G$. A sequence of rule applications is called a derivation.

Generated Languages

The language $L(G)$ generated by a grammar $G = (N, \Sigma, P, S)$ contains exactly those words that consist only of terminal symbols and can be derived from the start symbol with a finite number of steps:

$$L(G) := \{w \in \Sigma^* \mid S \rightarrow_G^* w\}$$

$\rightarrow_G^*$ denotes the arbitrary application of the productions in G , i.e., the reflexive-transitive hull of the transition relation $\rightarrow_G$

Example: $G = (N, \Sigma, P, S)$ mit $N = \{S, A, B\}$, $\Sigma = \{a, b\}$ and the following rules:

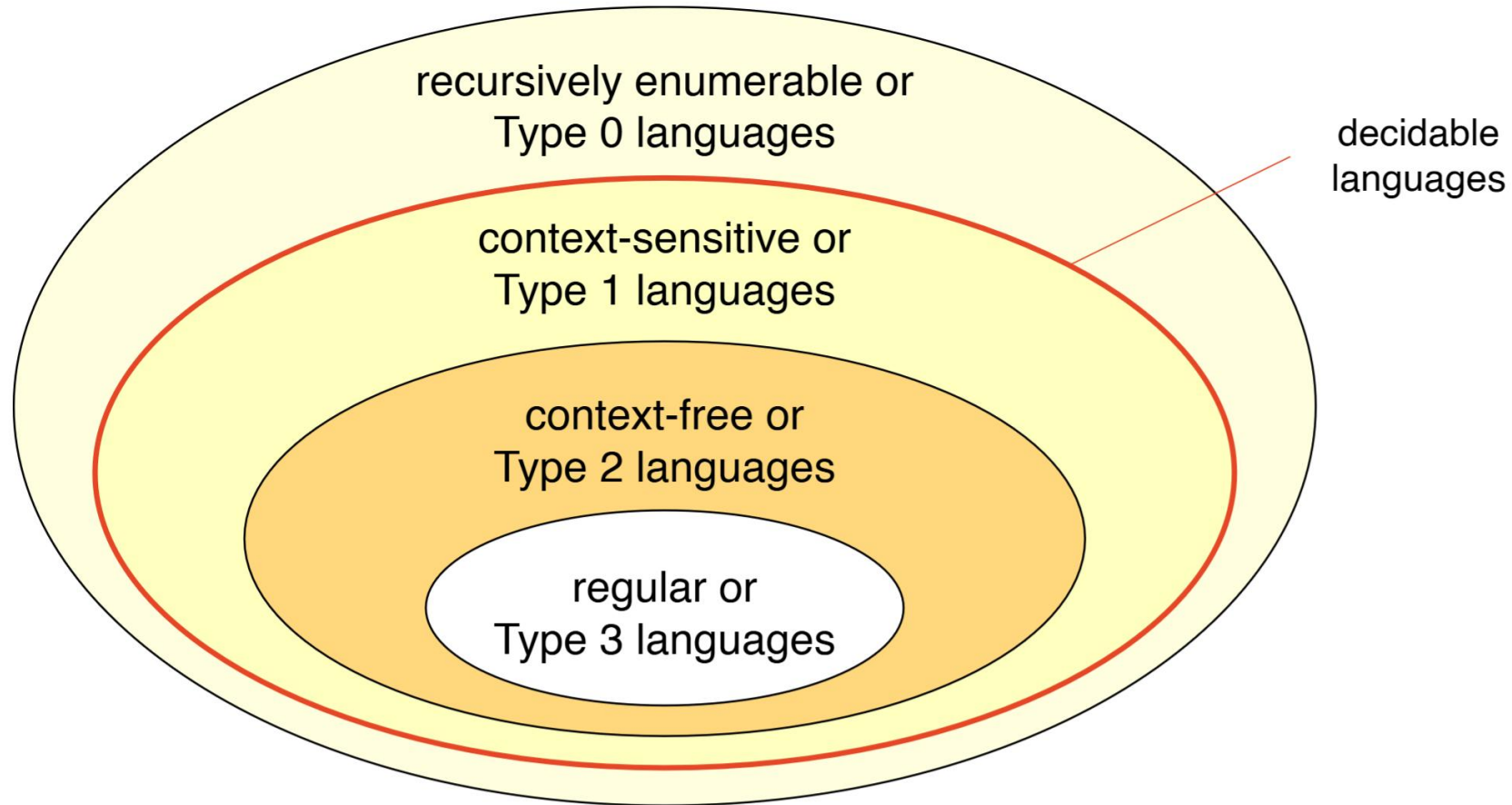
$S \rightarrow ABS$	$BA \rightarrow AB$	$Ab \rightarrow ab$
$S \rightarrow \varepsilon$	$BS \rightarrow b$	$Aa \rightarrow aa$
	$Bb \rightarrow bb$	

The Chomsky Hierarchy

Grammars are divided into four classes based on the complexity of the languages they generate.

- Type 0 ~ recursively enumerable: There are no restrictions for the rules in P .
- Type 1 ~ context-sensitive: For all rules $w \rightarrow w' \in P$ holds: $|w| \leq |w'|$
- Type 2 ~ context-free: For all rules $w \rightarrow w' \in P$ holds: w is a single variable; i.e., $w \in N$
- Type 3 ~ regular: The grammar is of Type 2, and for all rules, $w \rightarrow w'$ additionally holds: $w' \in (\Sigma \cup \Sigma N^*)$, i.e., the right-hand sides of the rules consist either of a terminal symbol or of a terminal symbol followed by a non-terminal.

The Chomsky Hierarchy



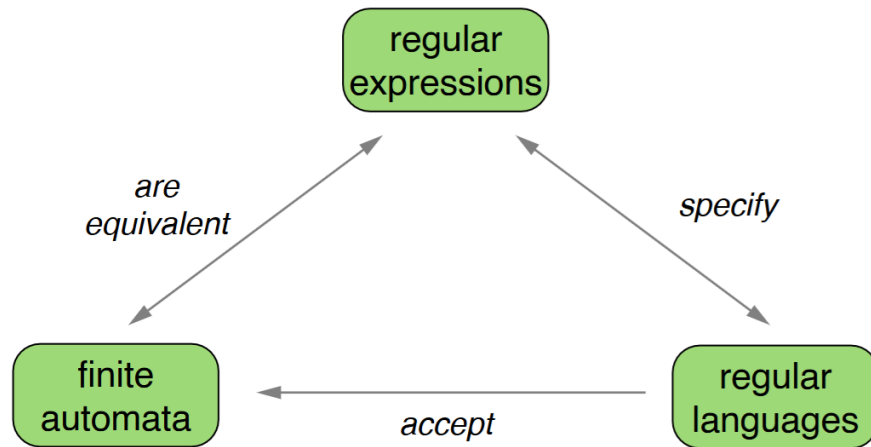
Remarks

- The Chomsky hierarchy represents a hierarchy with genuine subset relationships: $\text{Type 3} \subset \text{Type 2} \subset \text{Type 1} \subset \text{Type 0}$
- All languages of types 1, 2, or 3 are decidable:→ The decision problem if a word belongs to a language is decidable for all languages of Type 1, 2, or 3.→ There is an algorithm that, given a grammar G and a word w , decides in finite time if $w \in L(G)$ or not.
- The set of Type 0 languages is identical to the set of recursively enumerable or semi-decidable languages. Therefore, there are Type 0 languages that are not decidable. There are countably infinite grammars for generating a recursively enumerable language (Type 0)
- In compiler theory and natural language processing, Type 3 languages and grammars (lexical analysis, tokenization) and Type 2 languages and grammars (syntactic structure analysis) play a central role.

Calculi for Regular Languages

Different calculi for generating words in a regular language:

- a) Finite acceptor or automaton
- b) Regular expressions
- c) Type 3 grammar
- d) specifying finitely many equivalence classes



Types of Grammar: Summary

	Language generation calculi
Type 0	Type 0 grammar Turing machine
Type 1	context-sensitive grammar linear bounded Turing machine [Wikipedia]
Type 2	context-free grammar pushdown automaton
Type 3	regular grammar (Type 3 grammar) deterministic/non-deterministic finite automaton regular expression

	Complexity of the word problem [Wikipedia]
Type 0	undecidable
Type 1	exponential complexity, NP-hard
Type 2	$O(n^3)$
Type 3	linear complexity



UNIVERSITÄT
LEIPZIG

Questions?



UNIVERSITÄT
LEIPZIG

Regular Expressions

Regular Expression Syntax

A regular expression R can be composed recursively as follows, where F and G denote regular expressions:

	R	Language $L(R)$	Semantics
1.	a	$\{a\}$	The letter a
2.	FG	$\{fg \mid f \in L(F), g \in L(G)\}$	Concatenation
3.	$F \mid G$	$\{f \mid f \in L(F)\} \cup \{g \mid g \in L(G)\}$	Alternation
4.	(F)	$(L(F))$	Grouping
5.	F^+	$\{f_1 f_2 \dots f_n \mid f_i \in L(F), n \geq 1, i = 1, \dots, n\}$	Non-empty sequence of words from $L(F)$
6.	F^*	$\{\varepsilon\} \cup L(F^+)$	Arbitrary sequence of words from $L(F)$
7.	F^n	$\{f_1 f_2 \dots f_n \mid f_i \in L(F), i = 1, \dots, n\}$	Sequence of n words from $L(F)$
8.	ε	$\{\varepsilon\}$	The empty word

Examples

Every word in the language of this regular expression consists of two ones, followed by any number of zeros or ones, followed by two zeros.

Concatenation (5 times)

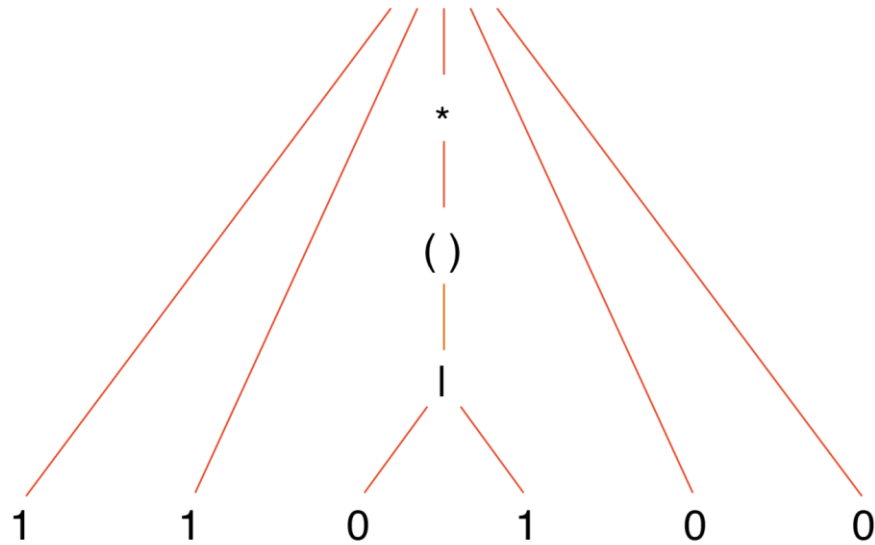
1 1 (0 | 1)* 0 0

Sequence

Bracketing

Alternative

Symbol



More Examples

R	Name of language $L(R)$	Words from $L(R)$
$(a \mid b) (c \mid d \mid \varepsilon)$	<i>Abc</i>	ac, bc, ad, bd, a, b
Dear($\varepsilon \mid \text{est}$) (Sir $\mid$ Madam)	<i>Salutation</i>	Dear Sir
$0 \mid 1 \mid \dots \mid 9$	<i>Digit</i>	7
$a \mid b \mid \dots \mid z$	<i>sLetter</i>	x
$A \mid B \mid \dots \mid Z$	<i>cLetter</i>	B
<i>sLetter</i> $\mid$ <i>cLetter</i>	<i>Letter</i>	m, N
<i>Letter</i> (<i>Letter</i> $\mid$ <i>Digit</i>) [*]	<i>Label</i>	Maximum, min7, a
<i>Digit</i> ⁺ . <i>Digit</i> ²	<i>MoneyAmount</i>	23.95, 0.50
(<i>cLetter</i> $\mid$ <i>cLetter</i> ² $\mid$ <i>cLetter</i> ³)– (<i>cLetter</i> $\mid$ <i>cLetter</i> ²)– (<i>Digit</i> $\mid$ <i>Digit</i> ² $\mid$ <i>Digit</i> ³ $\mid$ <i>Digit</i> ⁴)	<i>LicensePlatesDE</i>	PB–AS–0815
$1^3 (1 \mid 0)^* 0^3$	<i>Dual</i>	1111000, 1111101010000

Regular Expressions in Practice

An important use of regular expressions in languages used for natural language processing is the specification of text patterns.

Example: Display of all file names of the form

“winter-term (0|1|2|3|4|5|6|7|8|9)².html”

- Unix-Shell: `ls winter-term[0-9][0-9].html`
- Php: `$d = "[0-9]"; preg_match("/winter-term$d$d\.html/", $files)`

Remarks

- If names of regular expressions are used in other regular expressions, they must be identified as part of the meta language.
- Each scripting language for text processing uses a different syntax to specify regular expressions; the construction principles and power are comparable.
- The specification of regular expressions in PHP is taken from the Perl scripting language.

The Exact Matching Problem

Given a string R called the pattern and a longer string T called the text, the exact matching problem is to find all occurrences, if any, of pattern R in text T .

Example:

- Given $R = \text{aba}$ and $T = \text{bbabaxababay}$, then R occurs in T starting at locations 3, 7, and 9.
- Two occurrences of R overlap.

Importance of the exact matching problem:

- Exact matching is solved for searching local files and data.
- Main bottleneck is the speed with which the data can be read into memory.
- Speed rests on “preprocessing” pattern R (or text T) to enable skipping during search.

Perl Compatible Regular Expressions

Many programming languages (e.g. PHP, Python, Java) use a Perl-like syntax for regular expressions

*"**Delimiter** **Regular_Expression** **Delimiter** [**Flags**]"*
Pattern

- The delimiter must be a non-alphanumeric character.
- Optional flags influence the matching strategy. [pcre.org, PHP, Python, JavaScript, Java, .NET]
- Search and replace functions with regular expressions

Perl Compatible Regular Expressions

A regular expression R can be composed recursively as follows. Let F and G denote regular expressions and $L(F)$, $L(G)$ their languages over the alphabet Σ .

R [pcre.org]	Semantics
1. a	The letter a
$.$	Any symbol from Σ (except line break)
$[\Sigma']$	Character class: Any symbol from $\Sigma' \subseteq \Sigma$
$[\wedge\Sigma']$	No symbol from character class $\Sigma' \subseteq \Sigma$
$\backslash\Sigma'$	Escaped letter from a controlled $\Sigma' \subset \Sigma$, denoting a character class
2. FG	Concatenation
3. $F \mid G$	Alternation
4. (F)	Grouping or capturing
$F?$	F is optional (same semantics as $F \mid \varepsilon$)
5. F^+	Non-empty sequence of words from $L(F)$
6. F^*	Arbitrary sequence of words from $L(F)$
7. $F\{n\}$	Sequence of n words from $L(F)$
$F\{m, n\}$	Sequence with at least m and at most n words from $L(F)$
$^$ or $$$	Assertion: Beginning or end of the text string (nothing may precede or follow it)

PCRE: Character Classes

Character classes match a single token from a list of characters.

<i>R</i>	Semantics	Example
[...]	Positive character class matches listed character	[aeiou] matches vowels
[^...]	Negative character class matches only unlisted characters	[^aeiou] matches anything except for vowels
[<i>x</i> – <i>y</i>]	Range: matches any letter between <i>x</i> and <i>y</i> inclusive	[a-zA-Z] matches alphabetic characters
\d	Matches any decimal digits. Short for [0–9]	
\D	Matches all except decimal digits. Short for [^0–9]	
\s	Matches white space characters. Short for [\t\n\r\f\v]	
\S	Matches all except white spaces. Short for [^\t\n\r\f\v]	
\w	Matches alphanumeric characters. Short for [a-zA-Z0–9]	
\W	Matches non-alphanumeric characters. Short for [^a-zA-Z0–9]	

Remarks

-] needs to be escaped inside character classes to be matched: `[^]]` does not match a but, e.g., `]]`.
- The range in a character class denotes a series of codepoints. As such, `[-2]` is valid and identical to `[-./012]`. `[A-z]` matches not only alphabetic characters.
- Character classes cannot be nested but `\d`, `\D`, `\s`, . . . can be used inside character classes:
 - `^[a-z][A-Z]` matches the regex `^[a-z]`, `[A-Z]`, and `]` in sequence.
 - `^[^s\w]` matches any character that is neither a space, nor a word-character.

PCRE: Groups and Backreferencing

Groups give precedence to subformulas and can store (capture) matches for later reference.

<i>R</i>	Semantics	Example
(F)	Capture Group: Groups F and stores the matched value	$(ab)^+$ matches $ab, abab, \dots$
$(?:F)$	Non-capturing group: Groups F	$(?:ab)^+$ matches $ab, abab, \dots$
$(?<name>F)$	Named capture group: Groups F and stores it under a $name$	
$\backslash n$	Backreference the n -th capture group	$(\backslash w+)^+ \backslash 1$ matches <u>fully reduplicated words</u>
$\backslash k\{name\}$	Backreference capture group named “ $name$ ”	$(?<word>\backslash w+)^+ \backslash k\{word\}$ is the same as above

- Capture groups store the last match: The regex $([ab])^+\backslash 1$ matches abb but not aba .
- Named capture groups can extract information from different positions in the regex:
the regex $\backslash d\backslash d.(?<month>\backslash d\backslash d).\backslash d\backslash d|(?<month>\backslash d\backslash d)\backslash d\backslash d\backslash d$ extracts the month from $16.01.70$ and $01/16/70$.

Remarks

- The `\n` syntax is highly ambiguous:
 - If `n` is an octal number (`\1`, `\2`, . . . , `\7`, `\10`, . . .), it matches the character point with value `n`.
 - If `n` is a single non-octal digit (`\8`, `\9`), it matches the respective character.
 - If `n` is a valid index for a capture group, the above is irrelevant and the capture group is backreferenced. `\8` matches the character 8 but `()()()()()()()\8` only matches the empty word. `\49` gives an error because there is no 49th capture group but `\50` matches `(`.
- PCRE introduces `\gn` or `\g{n}` to reference capture groups and `\on` for octal codepoints. These are less common in other flavours of regex.
- Regular expressions compatible with PCRE2 are more powerful than regular grammars. The language $L = \{ww \mid w \in \{0, 1\}^*\}$ is not context free but can be decided by the regex `([01])*\1`.

PCRE: Quantifiers

Quantifiers allow for repeated matching of a regular expression F .

R	Semantics	Example
$F\{m\}$	Sequence of m words from $L(F)$	<code>\d{2}</code> matches exactly two digits
$F\{m, \}$	Sequence of at least m words from $L(F)$	<code>\d{2, }</code> matches two or more digits
$F\{m, n\}$	Sequence of at least m and at most n words from $L(F)$	<code>\d{2, 5}</code> matches two to five digits
$F\{, n\}$	Sequence of at most n words from $L(F)$ (<i>since PCRE2 10.43</i>)	<code>\d{, 10}</code> matches up to ten digits
$F?$	Equivalent to $F\{0, 1\}$	<code>\w?</code> matches at most one word-character
F^*	Equivalent to $F\{0, \}$	<code>\w*</code> matches any number of word-characters
F^+	Equivalent to $F\{1, \}$	<code>\w+</code> matches one or more word-characters

Default is greedy matching: as much as possible and backtracking until a match is found.

`\d*\d\d` tries to match **1234** but `\d\d` cannot be matched

`\d*\d\d` tries to match **1234** but `\d` cannot be matched

`\d*\d\d` tries to match **1234** and but `\d\d` matches as well → success!

PCRE: Quantifiers

Quantifiers allow for repeated matching of a regular expression F .

R	Semantics	Example
$F\{m\}$	Sequence of m words from $L(F)$	$\backslash d\{2\}$ matches exactly two digits
$F\{m, \}$	Sequence of at least m words from $L(F)$	$\backslash d\{2, \}$ matches two or more digits
$F\{m, n\}$	Sequence of at least m and at most n words from $L(F)$	$\backslash d\{2, 5\}$ matches two to five digits
$F\{, n\}$	Sequence of at most n words from $L(F)$ (<i>since PCRE2 10.43</i>)	$\backslash d\{, 10\}$ matches up to ten digits
$F?$	Equivalent to $F\{0, 1\}$	$\backslash w?$ matches at most one word-character
F^*	Equivalent to $F\{0, \}$	$\backslash w^*$ matches any number of word-characters
F^+	Equivalent to $F\{1, \}$	$\backslash w^+$ matches one or more word-characters

Modify the matching behaviour by adding $?$ (lazy) or $^+$ (possessive) behind the quantifier

$d+?$ matches as few digits as possible, but at least one. $\backslash d^{++}$ matches greedily without backtracking

Strings	R			
	$" . * "$	$" . * ? "$	$" . * + "$	$" [^ "] * + "$
"hi"	$\{"hi"\}$	$\{"hi"\}$	$\emptyset$	$\{"hi"\}$
"foo" "bar"	$\{"foo" "bar"\}$	$\{"foo", "bar"\}$	$\emptyset$	$\{"foo", "bar"\}$

Remarks

- Possessive matching is more efficient than greedy or lazy matching because no backtracking is performed.
- Possessive matching is a special case of an atomic group (written `(?>F )`). Atomic groups cannot be backtracked.
 - The regex `(?>a|aa)a` matches `aa` but not `aaa` because for the latter it would initially match `a` for the capture group (the first matching option in the alternative) and then have to backtrack when the end of the string is reached.
 - What strings would `(?>aa|a)a` match?
- `F ?+`, `F *+`, and `F ++` are short for `(?>F ?)+`, `(?>F *)+`, and `(?>F +)+` respectively.

PCRE: Assertions

Assertions test if a regex would match without matching it.

<i>R</i>	Semantics
$(?=F)$	(Positive) lookahead: Asserts that F matches at the current location.
$(?!F)$	Negative lookahead: Asserts that F does not match at the current location.
$(?<=F)$	(Positive) lookbehind: Asserts that F matches right before the current location.
$(?<!F)$	Negative lookbehind: Asserts that F does not match right before the current location.
	$(?<=\$) (?<!\$) \backslash d^+$ matches any number in the text that is not preceded by a \$.
$^$	Begin of string anchor: Asserts the start of the input sequence.
$\$$	End of string anchor: Asserts the end of the input sequence.
	$^[01]^+\backslash\$$ checks that the input entirely represents a binary number
$\backslash b$	Asserts a word boundary. Short for $(?<=\backslash W) (?=\backslash w) (?<=\backslash w) (?=\backslash W)$
$\backslash B$	Asserts the opposite of $\backslash b$. Short for $(?<=\backslash W) (?=\backslash W) (?<=\backslash w) (?=\backslash w)$
	$\backslash b[Tt]he\backslash b$ matches the word “the” but not if it occurs inside another word as in “atheist”.
	$\backslash Bing\backslash b$ matches the suffix -ing. For example in “climbing” or “sing” but not in “singer”.

- $^$, $\$$, $\backslash b$, $\backslash B$ are simple assertions and $(?=F)$, $(?!F)$, $(?<=F)$, $(?<!F)$ are assertion groups.
- Simple assertions cannot be followed by quantifiers. For example, $\backslash b^+$ is illegal but $(\backslash b)^+$ is allowed.



UNIVERSITÄT
LEIPZIG

Questions?